Measuring Network Latency from a Wireless ISP: Variations Within and Across Subnets

Simon Sundberg
Karlstad University
Karlstad, Sweden
simon.sundberg@kau.se

Anna Brunstrom
Karlstad University
Karlstad, Sweden
anna.brunstrom@kau.se

Simone Ferlin-Reiter
Red Hat
Stockholm, Sweden
sferlin@redhat.com

Toke Høiland-Jørgensen
Red Hat
Copenhagen, Denmark
toke@redhat.com

Robert Chacón
JackRabbit Wireless
El Paso, Texas, United States
robert.chacon@jackrabbitwireless.com

Abstract

While Internet Service Providers (ISPs) have traditionally focused on marketing network throughput, it is becoming increasingly recognized that network latency also plays a significant role for the quality of experience. However, many ISPs lack the means to continuously monitor the latency of their network. In this work, we present a method to continuously monitor and aggregate network latency per subnet directly in the Linux kernel by leveraging eBPF. We deploy this solution on a middlebox in an ISP network and collect an extensive dataset of latency measurements for both the internal and external parts of the network. We find that our monitoring solution can monitor all subscriber traffic while maintaining a low overhead of only around 1 % additional CPU utilization. Our analysis of the latency data reveals a wide latency tail in the last-mile access, which grows during busy periods in the evening. Furthermore, we dissect the external network latency and uncover the latency profiles for the most popular autonomous systems.

CCS Concepts

• **Networks** → **Network monitoring**; **Network measurement**; **Network performance analysis**; **Middle boxes / network appliances**; *Deep packet inspection*; • **Information systems** → *World Wide Web.*

Keywords

passive measurements, network latency, RTT, wireless ISP, eBPF

ACM Reference Format:
Simon Sundberg, Anna Brunstrom, Simone Ferlin-Reiter, Toke Høiland-Jørgensen, and Robert Chacón. 2024. Measuring Network Latency from a Wireless ISP: Variations Within and Across Subnets. In *Proceedings of the 2024 ACM Internet Measurement Conference (IMC '24), November 4–6, 2024, Madrid, Spain.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3646547.3688438



This work is licensed under a Creative Commons Attribution International 4.0 License.

IMC '24, November 4–6, 2024, Madrid, Spain
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0592-2/24/11
https://doi.org/10.1145/3646547.3688438

1 Introduction

While Internet Service Providers (ISPs) and other network operators have historically focused on network throughput, network latency has been shown to be just as important for delivering a good Quality of Experience (QoE) [10]. For web browsing, it has been shown that increasing bandwidth past 5 - 20 Mbps makes little difference for the page load times [16, 29]. Latency has also been shown to have a large impact on the QoE across a wide range of other networked applications, like streaming video [7], and in particular live video [46], online gaming [26, 45] and web searches [3]. It should therefore be of great interest for network operators, like ISPs, to monitor the network latency on their networks.

While network operators often monitor the amount of traffic their networks process, accurately monitoring the network latency is more challenging. Active network latency measurements, such as pinging, is commonly used and can provide a general estimation of the network latency between two nodes. However, active latency measurements do not say anything about the network latency that normal application traffic on the network experiences. An ISP operator can periodically ping popular content providers from its core network to obtain the network latency to these destinations. However, the ping probes may not experience the same processing and queuing delays as normal application traffic, and due to load balancing may not even take the same routes or reach the same destinations. Therefore, ping measurements do not accurately reflect the latency of the application traffic [32]. Furthermore, pinging external locations from the network does not account for the latency in the access network, and previous work has found that, for broadband access, the last mile access often contributes to a large portion of the end-to-end latency users will experience [27, 31].

The network latency experienced by actual application traffic can, however, be inferred directly from the traffic itself by using passive monitoring approaches. Unfortunately, capturing full packet traces to analyze the traffic with tools like tcptrace [36] may not be feasible with the high packet rates encountered in modern networks. Luckily, several approaches to passively monitor latency on high speed networks by leveraging programmable networks have recently been presented, with Dart [35] using P4, and epping [41] and nooBpf [39] using eBPF. While these tools are able to capture highly granular network latency information, they are to the best of our knowledge not yet widely used to continuously monitor network latency of large scale live networks. One reason for this is

the sheer amount of information these tools may capture. Indeed, in [35] Dart is only demonstrated on a short 15-minute packet trace from just two campus subnets, yet still yields 12.78 million Round Trip Time (RTT) samples, corresponding to an average rate of over 14 000 RTT samples per second. Meanwhile, in [41] epping is shown to report over 100 000 RTTs per second in some scenarios. Collecting and storing such vast amounts of RTT samples with related metadata, like which flow the RTT is from, is not practical.

In this work, we therefore propose a method to efficiently aggregate the large amount of RTT samples these tools produce, while still retaining a large amount of useful information. With the goal to ultimately enable network operators to continuously monitor network latency for traffic traversing their networks, we make the following contributions:

- We implement aggregation of RTTs into per-subnet histograms as an extension to epping, using eBPF to perform the aggregation directly in the kernel as soon as the RTT is computed. We also extend epping to collect additional traffic features, such as the overall traffic distribution and ECN utilization. By providing the source code for this extension, we allow anyone to deploy continuous latency monitoring in their own networks.
- We deploy our epping extension inside a Wireless ISP (WISP) network, monitoring the latency for all incoming and outgoing traffic over a period of one month. In total, we observe over 356 billion packets and collect over 9 billion RTT samples, while only adding around 1 % additional CPU overhead. By making the dataset publicly available we allow other researchers and operators to further analyze the data and gain insight on how latency can vary in the wild.
- We perform an extensive analysis of the collected latency data to demonstrate the type of information network operators can gain from continuously monitoring their network latency. Among others, we find a large latency tail in the internal part of the network that varies over time, increasing during busy hours. In contrast, we find the latency tail on the external networks to be much narrower and relatively stable over time, although we also detect some temporary latency spikes in the upstream networks. Furthermore, we examine the latency distribution for the most popular Autonomous Systems (ASes). We show how the latency typically follows a multi-modal distribution with narrow peaks associated with the handful of data centers handling the traffic, although thin latency tails are still present across the network.

The remainder of the paper is structured as follows: Our extensions to the epping tool to facilitate lightweight continuous latency monitoring are introduced in Section 2. Section 3 describes our measurement setup and data collection at the WISP. The high level characteristics of the captured traffic is examined in Section 4, followed by an in depth analysis of how the external latency varies at different levels of granularity in Section 5. Section 6 covers how our work relates to previous measurement studies while Section 7 discusses some of the limitations with our work. Section 8 concludes the paper.

2 Passively Collecting RTTs

To passively collect large amounts of RTT measurements with minimal impact on the network, we build upon our previous work with epping [41], which provides efficient passive monitoring of TCP RTTs using eBPF. As we noted in [41], when monitoring large amounts of traffic epping's approach of directly reporting individual RTT values may not only result in a large amount of output, but also creates considerable overhead with frequent communication between kernel space and user space. To make epping suitable for long-term monitoring from an ISP vantage point, we therefore extend epping with an aggregated mode and make the source code publicly available[1]. In this aggregated mode, the eBPF programs aggregate the inferred RTTs, storing them in BPF maps kept in the kernel instead of directly pushing them to user space. The user space agent then periodically fetches the aggregated RTT entries, reporting the RTT distribution together with select metrics, such as the mean and 95th percentile, for the observed period.

In Section 2.1 we provide a brief summary of how the original epping works. We then describe how we address several key challenges for efficient in-kernel aggregation in Sections 2.2 to 2.5, and Section 2.6 finally describes additional features we add to epping.

2.1 An Overview of epping

The epping tool infers RTTs from existing TCP traffic by measuring the time between observing a data packet and the corresponding acknowledgment (ACK) in the return path. To match packets and ACKs, epping relies on the `TSval` and `TSecr` fields in the TCP header, provided by the TCP timestamp extension specified in RFC 7323 [5]. With TCP timestamps enabled, the TCP sender puts its own timestamp into the `TSval` field and the receiver echos the received timestamp from the `TSval` field in the `TSecr` field. The `TSval` value of a data packet can thus be matched against the ACK with a matching `TSecr` value. Matching TCP timestamps avoids the retransmission ambiguity that sequence and acknowledgment numbers suffer from. To avoid the overhead of traditional packet capturing, epping is implemented as a set of eBPF programs attached to the traffic control (`tc`) layer. The eBPF programs run each time a packet is received or transmitted by the Linux kernel, allowing them to monitor the packets directly inside the kernel network stack. Without our aggregation mechanism, the eBPF programs then push each inferred RTT value through a perf ring buffer to a user space agent for reporting. For a more detailed description of epping, we refer the readers to our previous paper [41].

2.2 Capturing the RTT Distribution

For the aggregation of RTT values to be beneficial, we have to find a way to represent the RTTs that is both compact and expressive enough to retain a large part of the useful information from the original values. Simple metrics of central tendency, such as moving averages, fail to provide an accurate picture of how network latency behaves. As latencies are often tail-heavy, and latency spikes can have a disruptive effect on the end user QoE, high percentiles are often used to study these tails. However, only capturing e.g. the 99th percentile may still miss important differences within the bottom 99 % of the latency. Furthermore, percentiles themselves cannot

[1] Available at https://github.com/xdp-project/bpf-examples/tree/master/pping

be further aggregated. That is, a set of percentiles from different periods or different parts of the network cannot be used to compute a percentile for a longer period or a larger part of the network.

To provide a comprehensive view of the monitored network latency, we want to capture the underlying latency distribution. We therefore chose to use histograms, which is a simple data structure well suited for capturing distributions. A wide range of metrics can be approximated from the histogram data, and multiple histograms can also be merged to create a more holistic view of the data. Furthermore, to allow an ISP operator to examine the latency to a specific AS, to a specific part of their own network, or even to a particular subscriber, we keep separate histograms per subnet of configurable size. In addition to the histogram, we also keep track of the minimum and maximum observed RTT for each subnet.

2.3 Runtime Overhead

As the RTTs are aggregated as soon as they are inferred in the epping eBPF programs, which run in-line with the kernel network stack, the runtime overhead for aggregating the RTTs must be kept low to avoid substantially slowing down the network. The histogram data structure is well suited for this, as it can be updated by a single division operation to calculate the bin index followed by incrementing the corresponding bin counter. However, to update the histogram we must first lookup the histogram for the correct subnet. Initially, we used a BPF Longest Prefix Match (LPM) trie map to lookup the subnet histogram from the IP address, but found that the LPM lookup became too slow when monitoring a large amount of subnets. We therefore use a BPF hashmap instead, where the subnet key is calculated by performing a bitwise `AND` between the IP address and a pre-calculated mask to zero out the last bits of the IP address. Furthermore, as the key size impacts the performance of BPF hashmaps [24], we keep separate maps for IPv4 and IPv6 traffic instead of mapping the IPv4 addresses into the IPv6 address space, allowing us to use smaller 32-bit keys for the commonly seen IPv4 traffic.

2.4 Concurrency

As the Linux network stack is multi-threaded and can process multiple packets concurrently on different CPU cores, another challenge to consider is that several instances of the epping eBPF program may also run in parallel and try to update the aggregated state concurrently. To allow safe concurrent updates without costly locking mechanisms to synchronize access across CPU cores, we utilize per-CPU BPF maps. With per-CPU maps, each subnet entry will maintain a separate copy of the histogram for each CPU core, allowing each CPU core to update its own copy of the histogram independently from the others. When the user space agent reports the RTT statistics, it will simply add the histograms from the different CPU cores together to obtain the complete statistics for the subnet. Furthermore, to avoid the user space agent from fetching and clearing the BPF maps while they are simultaneously updated by the eBPF programs, leading to inconsistent state, we use a double buffering scheme. We maintain two sets of the aggregation maps, one active set which the eBPF programs are currently updating, and one inactive set that the user space agent can fetch and clear. Whenever the user space agent wishes to fetch the current RTT statistics, the active and inactive sets of maps are switched, allowing the user space agent to fetch the current RTT statistics from the now inactive maps.

2.5 Memory Footprint

Another important aspect to consider is the memory footprint of the aggregated data structure. Our implementation favors low computational overhead at the cost of memory. In this work we configure the histograms to consist of 250 4 ms wide bins to capture the latency distribution up to 1 second. As each bin is realized through a 32-bit counter, each histogram takes up 1 KB of memory. Together with the per-CPU maps and double buffering, the aggregated data for a single subnet thus requires just over 2 KB of memory per CPU core, translating to 10s of megabytes per core to track thousands of subnets. We note that this memory footprint makes it unsuitable for memory constrained environments, such as programmable switches. This is, however, not a major concern for a general purpose Linux-server, which is the environment we target. At the cost of some additional computational overhead and complexity, future work could use histograms with non-uniform bin widths, such as HdrHistogram [42], circllhist [17] or UDDSketch [13], to capture the RTT distribution with a lower memory footprint.

2.6 Collecting Additional Traffic Features

While monitoring the network latency itself can provide useful information, additional context is often needed to interpret the measured latency. In addition to aggregating the TCP RTTs, we therefore also extend epping to collect several extra traffic features. On the per-subnet basis, we collect packet and byte counts of traffic received by and transmitted to the subnet, allowing the network latency to be correlated against traffic load and the importance of the measured latency for a particular subnet to be assessed. These traffic counts are broken up into three categories: TCP traffic with TCP timestamps, TCP traffic without TCP timestamps, and non-TCP traffic. As epping can currently only infer RTTs for TCP traffic with the TCP timestamp option enabled, this separation of traffic counts helps assess the fraction of traffic epping is able to monitor latency for, as we do in Section 4.1. We also add global byte and packet counters per protocol as indicated in the IP header [20], as well as counters for the four Explicit Congestion Notification (ECN) code points (Not-ECT, ECT(0), ECT(1), and CE).

3 Measurement Setup

We collect measurements from within the core network of the wireless ISP JackRabbit Wireless, operating in El Paso, Texas, USA. JackRabbit Wireless serves approximately 400 subscribers, of which around 95 % are households. In Section 3.1 we describe the network topology of JackRabbit Wireless, while Section 3.2 covers how we deployed epping inside the ISP network to perform the measurement campaign. We assess the overhead of our measurement setup in Section 3.3.

3.1 Network Topology

JackRabbit Wireless relies on fixed wireless connections for both access and backhaul networks, as illustrated in Figure 1. A client connects to one of JackRabbit Wireless' access points over a wireless

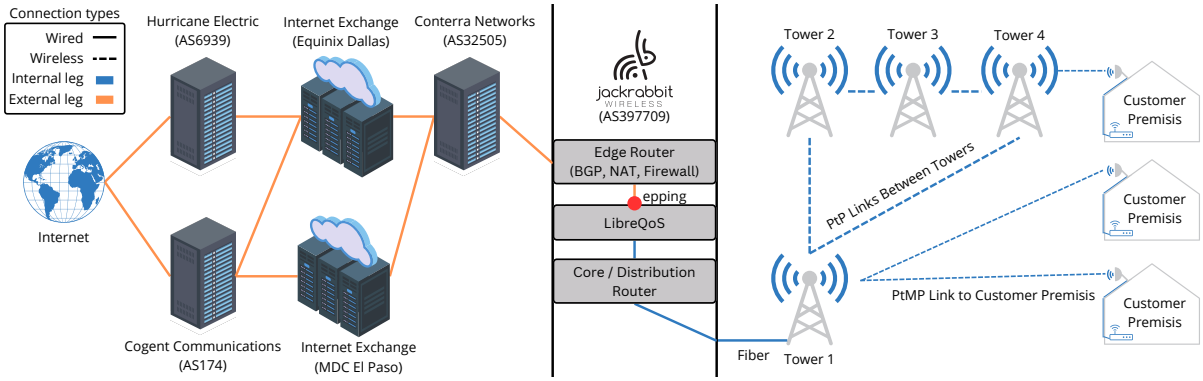


Figure 1: JackRabbit Wireless network topology. The red dot shows the observation point where epping was deployed.

link via an antenna installed at the client's premise. Each access point serves multiple clients using Point-to-Multi Point (PtMP) radios. The access network is mainly based on Ubiquiti's LTU platform, with LTU Rocket access points and LTU-LR client radios, operating in the 5GHz spectrum. Some sites also utilize 60 GHz millimeter wave. The access points backhaul the traffic over Point-to-Point (PtP) wireless links, which mainly use Aviat WTM4800 radios operating at 80 GHz or Ubiquiti AF60-LR operating at 60 GHz. In case rain disrupts the primary links, which operate in the millimeter wave spectrum, traffic is backhauled over ePMP 400C backup radios at 5 GHz.

Once traffic reaches the core router at JackRabbit Wireless' central site, it is routed through a middlebox running LibreQoS[2] [22]. LibreQoS is an open source QoE platform for ISPs, which applies Fair Queuing (FQ), Active Queue Management (AQM), and shaping to improve the QoE for all subscribers. After the LibreQoS middlebox, the traffic goes through the edge router, which performs Network Address Translation (NAT) and forwards the traffic to the upstream ISP, Conterra networks. Conterra networks in turn peers at a local Internet Exchange Point (IXP) in El Paso as well as at a more remote IXP in Dallas, which, in turn, transit via Cogent Communications and Hurricane Electric to provide global Internet connectivity.

3.2 Measurement Configuration

A single instance of the extended version of epping is set up on the LibreQoS middlebox. We monitor the interface facing the upstream networks, connected towards the edge router as illustrated in Figure 1. The RTTs are captured relative to the point epping is deployed. For downlink traffic, epping will thus capture the RTT on the internal leg of the network, i.e. between the LibreQoS box and the subscribers' endhosts, shown by the blue lines on the right side of the figure. For uplink traffic, we will instead capture the RTT on the external segment of the network, i.e. between the LibreQoS box and the remote end hosts the subscribers connect to, shown by the orange lines on the left side of the figure.

We chose the LibreQoS box as our vantage point because all customer traffic passes through it, allowing us to monitor all of the traffic from this singular point. Furthermore, the eBPF based epping tool could easily be deployed on the LibreQoS box, as it is a general-purpose Linux based computer using the Linux network stack together with eBPF to implement several other network functions. This made it convenient for the ISP operator to compile epping from the source code and set it up as a background service on the LibreQoS box using our included systemd unit files. An alternative would have been to deploy epping closer to the edge of the network, but this would have required deploying epping on multiple suitable Linux-devices close to the edge in order to still monitor all of the traffic; it would also have shifted most of JackRabbit Wireless' distribution and core network to be included in the external part of the latency.

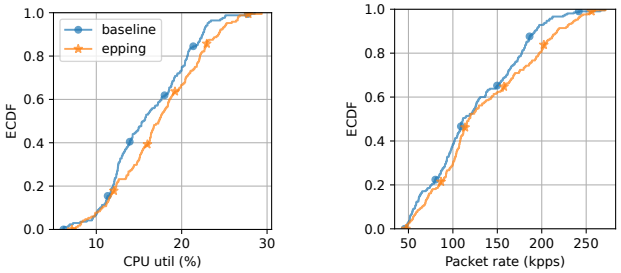
We configure epping to fetch aggregated data at 10 s intervals, where data is aggregated per /24 subnet for IPv4 traffic and per /48 subnet for IPv6 traffic. While more fine-grained per-IP data can be useful to locate issues in the internal network, we only collect data at the coarser subnet granularity to minimize the risk that any data can be associated with individual subscribers. Furthermore, we limit the capture to at most one RTT every 10 ms for any given flow, which is suitable in relation to the end-to-end latency in this setting and prevents bulk flows from being over-represented in the RTT statistics. We run epping over a one-month period, between 2023-10-30 to 2023-12-01, except for a 5 hour outage in the night of 2023-11-08. In total, we collect 357.6 million reports with per subnet statistics, observing 356.3 billion packets and capturing 9.025 billion RTT samples. As the aggregated data does not contain privacy sensitive information, we make the dataset publicly available[3] to allow other researchers to further explore and analyze the latency distributions as well as reproduce our results.

3.3 Measurement Overhead

While the ISP operator did not have any issues with the monitoring setup and did not notice any significant difference in load on the LibreQoS middlebox during the measurement campaign, we do not have any system load data from the measurement period. To assess the system overhead our measurement setup imposed on the

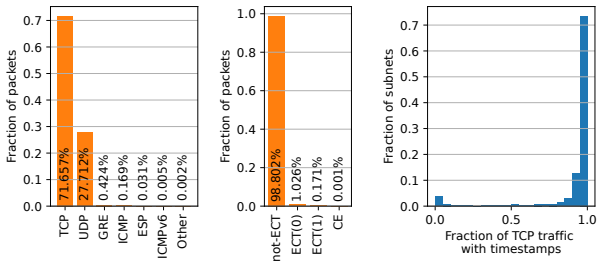
[2]The LibreQoS platform has also integrated parts of epping to sample and aggregate RTTs per subscriber. Work on further extending the RTT collection to for example aggregate RTTs per AS is currently ongoing.

[3]Dataset available at https://doi.org/10.5281/zenodo.13388093



(a) CPU (b) Packet rate

Figure 2: CPU utilization and traffic load without epping (baseline) and with epping.



(a) Protocols (b) ECN (c) TCP timestamps

Figure 3: Fraction of packets of each protocol and ECN codepoint, and fraction of TCP packets carrying TCP timestamps across the external subnets.

LibreQoS box, equipped with an AMD 3600X CPU, we asked the ISP operator to run the same measurement setup for an additional week and provide us with data on the system load during this period and a similar period without the measurements. The traffic load during these two weeks, around 130 kpps on average with peaks to around 270 kpps, is similar to the traffic load throughout the measurement period, and we therefore expect the measured overhead to be similar to the overhead imposed during the original measurement campaign.

The overall CPU utilization and traffic load for the 7-day period with epping running, and a baseline for a corresponding 7-day period without epping running, are summarized in Figure 2. While the CPU utilization in Figure 2a is overall slightly over one percentage point higher when running epping, the traffic load for the same period in Figure 2b also shows a comparable relative increase, resulting in very similar performance if the CPU utilization is normalized against the traffic load. At a high level, our measurement setup thus appears to impose negligible overhead, although we acknowledge that this simple comparison may not be entirely accurate as there may be various factors impacting the load during these two periods that we have not accounted for.

To get a more accurate measurement of the overhead and be able to estimate how the overhead of our aggregated version of epping will scale with traffic load, we also asked the operator to enable the collection of BPF runtime statistics in the Linux kernel (`kernel.bpf_stats_enabled=1`) for one day, which measures the number of times each eBPF program runs and their cumulative runtime. While this only includes the overhead of the eBPF programs, those are mainly the ones that will be impacted by higher traffic rates as they run for each processed packet, whereas the user space agent fetching and reporting the aggregated data operates at a fixed frequency. The epping program that processes ingress packets, i.e. downlink traffic in our setup, had an average runtime of 269 ns per packet across 10.8 billion invocations, whereas the egress program processed 5.4 billion packets with an average runtime of 325 ns per packet, giving a weighted average of 287 ns per packet in total. With the average packet rate in the network being around 130 kpps, this translates to around 3.7 % utilization of a single AMD 3600X core, or approximately 0.6 % when distributed across the 6 available cores. If the traffic load would be scaled up to 1 Mpps the overhead could then be expected to be around 5 % CPU utilization. While not necessarily negligible, this overhead is still low enough that it should be feasible for many smaller and medium sized ISPs to deploy similar measurement setups on their existing infrastructure, without the need for hardware upgrades or installing additional devices for the sole purpose of monitoring network latency.

4 Overall Traffic Distribution Characteristics

We first look at several aspects of the collected measurements as a whole, where Section 4.1 covers the protocol distribution, Section 4.2 the ECN utilization, Section 4.3 the overall distribution of the internal and external latency, and Section 4.4 how latency varies over time.

4.1 Protocol Distribution

As epping can currently only monitor latency for TCP traffic with TCP timestamps enabled, the protocol distribution impacts the subset of traffic for which RTTs can be inferred. Of the 356.3 billion observed packets, around 71.7 % of them carry TCP traffic, as illustrated in Figure 3a. The rest of the traffic is mainly UDP, which includes all traffic running on top of UDP, such as DNS and QUIC. Among the TCP traffic, we find that a whole 86.1 % of the packets have the TCP timestamp option enabled. The dependency on TCP timestamps does therefore not seem to be a major obstacle for using epping to monitor Internet traffic. In total, epping is able to monitor the latency for $0.7166 \times 0.8611 = 61.7$ % of the traffic during our one-month measurement campaign.

The fraction of TCP traffic with TCP timestamps, however, varies greatly between subnets, as shown by Figure 3c. For the vast majority of external subnets, over 90 % of the TCP traffic has timestamps. However, for around 4.7 % of the subnets, less than 10 % of the TCP traffic has timestamps, greatly limiting our ability to monitor latency for these networks, with only 0.04 % of the captured external RTTs covering these subnets despite them handling 6.8 % of all TCP traffic. We also note that TCP timestamps are generally disabled by default on Windows clients, but enabled by other popular Operating Systems (OS) such as macOS, iOS, Android and Linux [2]. Given that Windows is the most common OS on desktop computers, our measurements will likely largely lack RTTs from devices connected

Table 1: Number of TCP packets with TCP timestamps enabled and the number of RTTs that has been inferred from them, per traffic direction and IP version.

		Packets		Bytes		RTTs	
Link	IP	No. (Bil.)	Rel. (%)	No. (Tril.)	Rel. (%)	No. (Bil.)	Rel. (%)
Down	v4	131.99	59.69	185.31	83.34	5.48	60.73
	v6	16.74	7.57	22.37	10.06	0.53	5.85
Up	v4	66.83	30.22	13.11	5.90	2.78	30.79
	v6	5.59	2.53	1.56	0.70	0.24	2.63

via Ethernet, with most RTTs instead collected from portable or IoT devices connected over WiFi.

Takeaway. While epping's current reliance on TCP timestamps to infer RTTs leads to some observability gaps, notably for QUIC traffic and traffic from Windows clients, epping is still able to capture the latency for the majority of the traffic.

4.2 ECN Occurrences

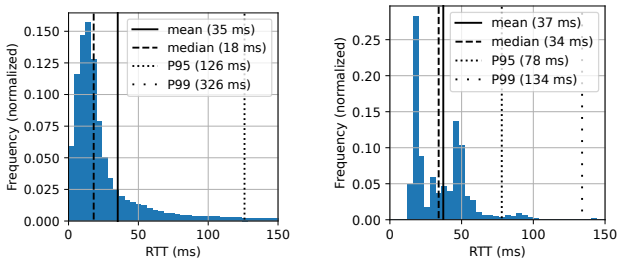
Figure 3b shows the frequency of the four different ECN code points in the IP header. While active measurements from recent studies [23, 33] indicate that ECN is widely supported by web servers, we note that only 1.20 % of the traffic in our measurements carry any type of ECN-marking. This is more in line with the passive measurements performed by Lim, H. et al. [23], where 7.52 % of the TCP connections from a campus network used ECN markings, although our measurements indicate even lower adoption.

Of the ECN-marked traffic, 85.6 % uses the ECT(0) mark for classical ECN-capable transports, while 14.3 % uses ECT(1), intended for L4S scalable congestion control algorithms [34]. With our passive measurements, however, we cannot determine if this ECT(1) marked traffic is indeed early deployments of L4S congestion controls, classic ECN capable transports using the previously equivalent ECT(1) code point instead of ECT(0), or network elements incorrectly remarking ECT(0) as ECT(1) packets. Furthermore, only 0.05 % of the ECN-marked packets had the CE code point set, used to indicate congestion. As we did not track ECN Echo in the transport headers, we cannot determine to which degree these infrequent congestion signals were actually echoed back to the sender. Future work could further extend epping to allow for a more detailed study of ECN markings in the wild.

Takeaway. In our data, ECN adoption is very low, and rarely used to signal congestion even when enabled.

4.3 Internal and External Latency Variation

In total, we captured 9.03 billion RTTs, with 66.6 % of them covering the internal network (between the capture point and the subscribers' devices) and 33.4 % covering the external networks (between the capture point and the Internet). A larger fraction of the RTTs being from the internal network is expected, given that most of the traffic is downlink. The relationship between the number of RTTs on each network segment is similar to the number of TCP packets with TCP timestamps going in each direction, as shown in Table 1. Most



(a) Internal **(b) External**

Figure 4: RTT distribution on internal and external segments of the network.

of the traffic is IPv4, with only around 8.5 % of the RTTs being for IPv6 traffic.

The aggregated histograms for all captured RTTs on the internal and external segments of the network are shown in Figure 4. Note that to calculate approximate summary statistics from the RTT histograms, e.g. mean and percentiles, we assume that all RTTs are in the center of their respective 4 ms bins, so that the real value may differ by ±2 ms. The mean value may also be underestimated, due to RTT values exceeding 1000 ms being truncated to the 0–1000 ms range the histograms cover. As expected, the internal network segment in Figure 4a has a low baseline latency, as the traffic only traverses the local network, with over a third of the RTTs being below 12 ms and the peak of the distribution being around 14 ms. The distribution is, however, quite heavy-tailed, with the mean latency being more than twice as high as the median, and a 99th percentile of 326 ms. Furthermore, roughly 0.16 % of the RTTs exceed 996 ms, putting them in the final half-open histogram bin. A small subset of the traffic thus experiences long delays in the internal network, which would be disruptive for any interactive applications.

For the external segment in Figure 4b, the baseline RTT is significantly higher, with few RTTs below 12 ms. On the other hand, the external latency tail is also much shorter, with a 99th percentile of only 134 ms. Furthermore, the external RTTs appear to have a multi-modal distribution, with two clear peaks around 18 ms and 46 ms, and a smaller peak around 30 ms, with relatively low variation around these peaks. These peaks correspond well with the measured latency to several of the nearest Content Delivery Network (CDN) and IXP locations: The RTT measured from the WISP operator's core router to the Dallas IXP match the 18 ms of the first peak, while the RTT to both Seattle - Washington (west coast) and Ashburn - Virginia (east coast), is around 46 ms. We further investigate the RTT distribution for these external subnets in Section 5. This multi-modal distribution causes the overall median of the external latency to be considerably higher than that of the internal latency. However, for the large fraction of traffic around the 18 ms peak, around 42 % of all external RTT samples, the external latency is going to be comparable to, and may in many cases be lower than, the internal latency.

We also briefly look at how the internal and external latency distributions for IPv4 and IPv6 traffic differ, with the empirical CDFs

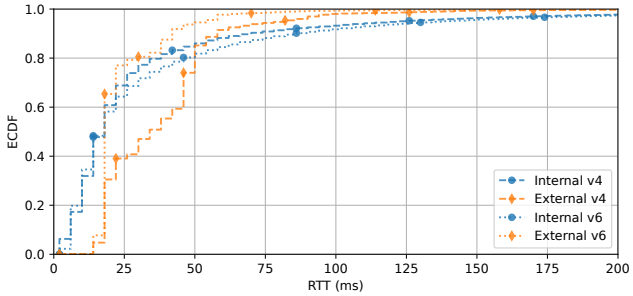


Figure 5: RTT distribution of internal and external segments of network, separated by IP version.

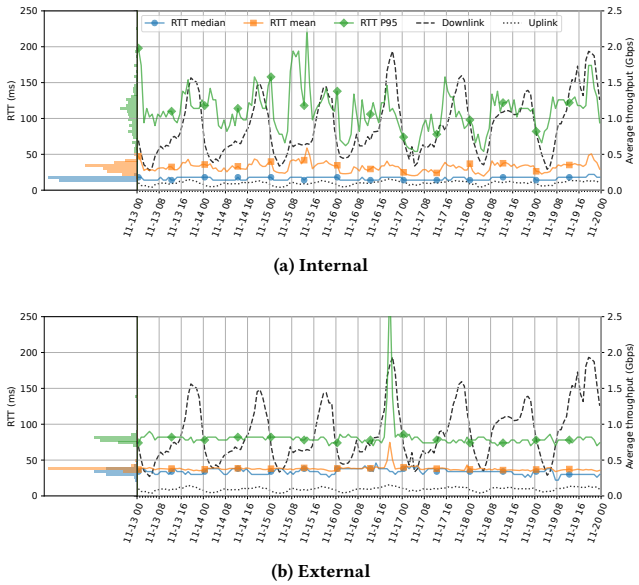
of the 4 distributions being displayed in Figure 5. The internal latency is similar for both IPv4 and IPv6, although IPv6 has a slightly wider latency tail than IPv4. This may be explained by the hardware used in JackRabbit's routing infrastructure having some IPv4 specific optimizations, as well as the IPv6 distribution likely being somewhat skewed by numerous IoT devices, which commonly use the contested 2.4 GHz WiFi band, primarily using IPv6 addresses. For the external networks, the IPv6 traffic also has a multi-modal distribution, but the vast majority of the traffic lies in the initial 18 ms peak, indicating it is largely served at the nearby Dallas IXP, resulting in the external latency overall appearing significantly lower for IPv6 than IPv4.

Takeaway. Despite JackRabbit Wireless adopting techniques such as FQ, AQM, and shaping in their network, the internal leg of the connections still has a long latency tail and makes up a significant part of the overall end-to-end latency. Further reducing last mile latency can thus still considerably improve the subscriber's QoE.

4.4 Latency Variation Over Time

Next we look at how latency varies over time. To clearly visualize the diurnal behavior and avoid excessively compressing the x-axis, we only show one week from the middle of the measurement period in Figure 6. However, we note that the other weeks show similar characteristics. We also include information on the traffic load in the same figure to examine its potential correlation with latency. The uplink and downlink traffic loads show a very clear diurnal behavior, where the hourly average reaches upwards of 2 Gbps around 18:00 - 22:00 in the evening, and then drops down to a few hundred Mbps between 02:00 - 05:00 in the morning. The mean and 95th percentile internal RTTs in Figure 6a clearly vary over time, while the median appears more stable. However, we note that the 4 ms resolution of the aggregated RTTs is quite coarse relative to the median value of approximately 18 ms, and we can therefore not detect small changes in the median latency. Furthermore, the 95th percentile, and to some extent the mean, of the internal RTTs appear to be partly correlated to the traffic load, where they are also higher during the afternoon and evening, and lower during the night and early morning.

In contrast, the external RTTs in Figure 6b are generally relatively stable over time, not showing any major variation with traffic load or time of day. There is, however, a large spike in tail latency in the



(a) Internal

(b) External

Figure 6: RTT (left y-axis) and average throughput (right y-axis) at 1 hour intervals over one week.

evening of November 16. While not directly visible in Figure 6b, further analysis reveals that this roughly 100 minute long spike originates from the latency in a large number of Amazon subnets suddenly increasing to between 250–320 ms before returning to normal again. We note that Cloudflare Radar[4] recorded a spike in Denial of Service (DoS) traffic towards the two affected Amazon ASes that coincides with this latency spike, which could offer an explanation for the underlying cause of the latency increase. There are a few more of these temporary latency spikes throughout the measurement period, including a similar spike at the same time the previous week. While outside the direct control of the ISP, we note that detecting these types of temporary spikes in external latency still provides useful information to the ISP operator, as it explains the poor performance the ISP's subscribers may experience when using the affected services during this period. Furthermore, if the external latency does not return to normal, it may prompt the ISP to contact upstream providers to resolve the issue.

To further investigate the potential correlation between network latency and traffic load, we compute the Spearman correlation between several different RTT metrics and the traffic load across the entire measurement period using the 10 s intervals the data was captured at. The results are shown as a heatmap in Figure 7. There are essentially three groups in Figure 7: The traffic load, the internal RTT, and the external RTT. As can be expected, metrics within the three groups are generally strongly correlated with each other, i.e. uplink and downlink traffic is strongly correlated, as is the amount of traffic in both bytes and packets, while most RTT metrics generally rise together when the RTTs increase. However, the correlations between uplink traffic measured in bytes and the

[4]See the spike at Nov. 17th 1:30 UTC at https://radar.cloudflare.com/security-and-attacks/as16509?dateStart=2023-11-16&dateEnd=2023-11-17

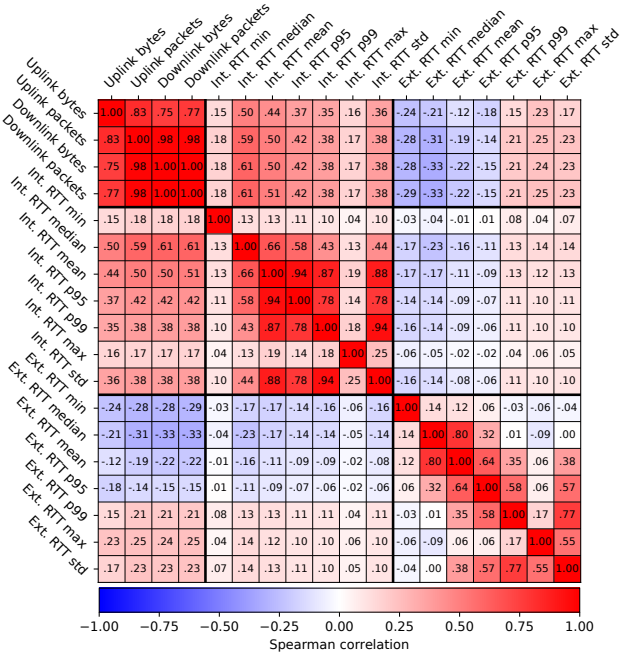


Figure 7: Spearman correlation matrix of several traffic load and RTT metrics computed at 10 second intervals.

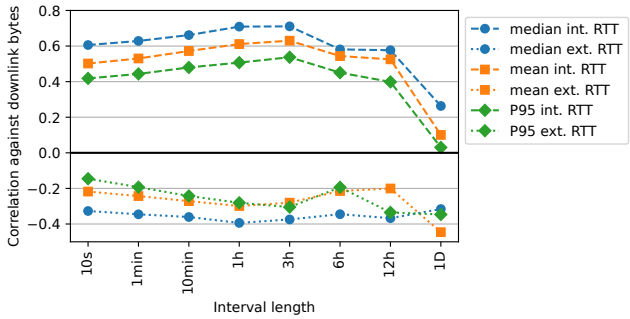


Figure 8: The Spearman correlation between several RTT metrics and downlink traffic in bytes, computed at various time resolutions ranging from 10 seconds to 1 day.

other traffic load metrics are a bit lower than other correlations within the traffic load group. This can be explained by the uplink traffic in bytes making up a fairly small fraction of the total traffic (see Table 1), and the size of individual uplink control or request packets not necessarily being tightly correlated to the size of the response in the downlink.

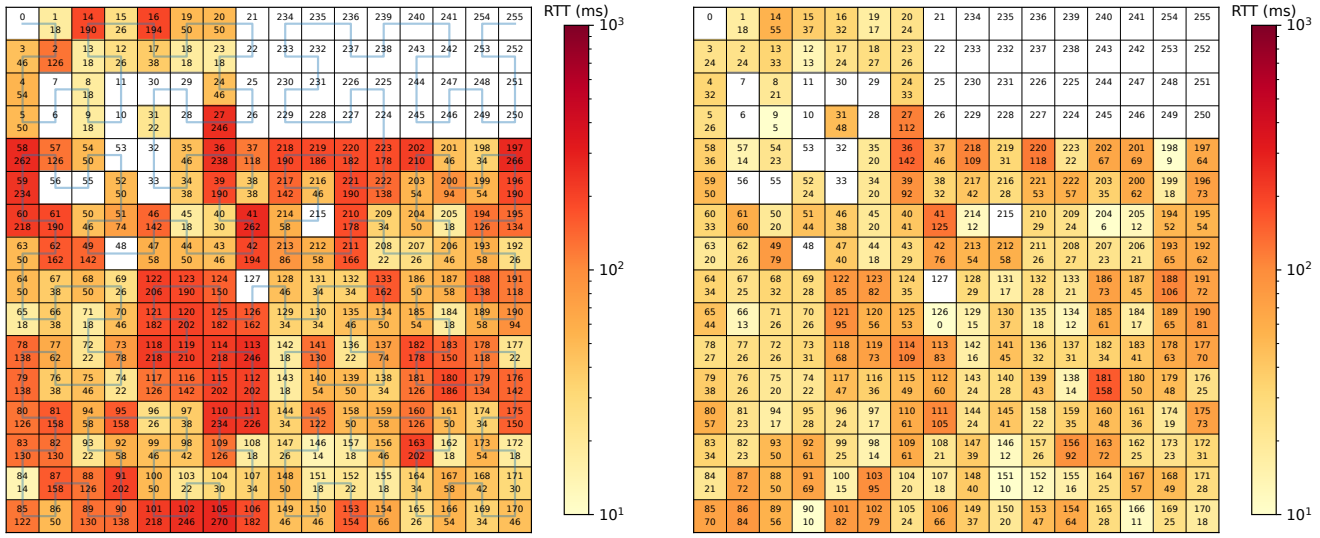
When it comes to the correlation between RTT and traffic load, the internal RTT values do indeed show some weak to moderate correlation with traffic load. It is worth noting that the correlation strengths show the opposite relation between median, mean, and 95th percentile RTTs compared to the magnitude of variation in Figure 6a. The median RTT thus increases more consistently with traffic load than the noisier mean and 95th percentile, even though the increase is relatively small. The external RTTs on the other hand show much weaker correlation to the traffic load, and many of the RTT metrics, like median and mean, actually seem to show a slight decreasing trend during periods with higher traffic loads. While the negative correlation is fairly weak, a slight decrease in median and mean RTT during the busy evening periods can likely be explained by the traffic shifting towards content that is served by nearby CDNs (e.g. Netflix) instead of applications used during work hours (e.g. remote VPNs on the east coast). It is also possible that various network caches become more efficient during high traffic loads, being able to serve a larger fraction of the traffic directly from cache.

We also compute the correlations between the RTT metrics and the traffic load at several time intervals larger than 10s, with the results for the median, mean, and 95th percentile of the internal and external RTTs shown in Figure 8. As can be seen, the correlation strength increases as the considered interval increases up to 3 hours, and then drops when using even coarser time scales. This suggests that the internal RTT is not necessarily tightly coupled to the exact traffic load on short timescales, but rather with longer periods where the average network load is high in general. This conclusion is also supported by the RTT peaks in Figure 6a often occurring at different times than the traffic peaks, even if the RTT as a whole, just like the traffic load, still clearly increases during the day and decreases during the night.

We note that with our singular measurement point and lack of detailed per-IP data we cannot pinpoint where in the internal network the latency increases during busy periods. While a correlation between latency and traffic load can often be an indicator of queuing latency, i.e. bufferbloat, the WISP operator employs FQ and AQM techniques that should largely address these issues in the access network, although bufferbloat in end user equipment is still possible. Another explanation for the latency increase may be provided by the wireless technologies used in the internal network: During busy hours, the radio spectrum is more congested, with the wireless links facing more interference and concurrent access. This in turn leads to higher probability of packet errors and collisions, which are translated into more retransmissions and backoff periods on the link layer and, thus, additional latency.

As the WISP operator relies on a wireless access network, a congested radio spectrum could negatively impact the latency in the WISP network. Bad weather could potentially also impact the latency, although there was almost no precipitation during our measurement period. The most likely culprit, however, is the WiFi access inside customer premises. As noted in Section 4.1, the RTTs are mainly collected from portable devices connected via WiFi, and many previous works [12, 19, 27, 31, 37, 40] have found that WiFi can cause large latency tails similar to the ones we observe on the internal network when the WiFi link becomes the bottleneck link or when multiple devices are competing for access. Indeed, the WISP operator mainly uses its own per-subscriber RTT tracking to identify subscribers with poor latency and help them improve their WiFi setup to provide a better quality of experience.

Takeaway. Internal network latency is variable and partly correlated to the overall network load, although not necessarily for



(a) Median

(b) Standard deviation

Figure 9: RTT per /8 IPv4 address block. The blocks follow a Hilbert curve, shown by the faint blue line in Figure 9a, with 0.0.0.0/8 in the upper left corner and 255.0.0.0/8 in the upper right corner. The upper number in each block shows the number of the block (e.g. 42 is 42.0.0.0/8), and the lower value shows the RTT in ms.

moment to moment fluctuations. This can result in subscribers experiencing different qualities of service depending on when they use the network. External network latency is more stable, although issues in upstream networks can cause temporary disruptions. The continuous latency monitoring can detect and inform the ISP operator about such events.

5 Variations in the External Latency

As noted in Section 4.3, the external RTTs have a multi-modal distribution. To better understand the behavior of the RTTs on the external subnets, we further break down the external RTTs at various levels of granularity. We first consider the latency at the coarse level of /8 IP address blocks in Section 5.1, and then dig into how the latency varies across the individual /24 subnets in Section 5.2. Finally, we look at the latency towards popular ASes in Section 5.3.

5.1 Variations Between /8 Address Blocks

We first visualize the RTT for each /8 address block across the entire IPv4 address space in Figure 9, inspired by the xkcd "Map of the Internet" [30]. We only consider IPv4, as the IPv6 address space is very large, and we have only observed traffic to a few sparse points in the IPv6 space. In general, /8 blocks are too large to show latency to specific services, although a few /8 subnets are allocated for single organizations, e.g. 12.0.0.0/8 is allocated to AT&T, 17.0.0.0/8 to Apple, and 38.0.0.0/8 to Cogent Communications. The RTTs to these three examples are generally fairly low, which is unsurprising, given that as major US-American based companies, they are likely to have Points of Presence (PoPs) close to the WISP, in Texas, from where we conduct the measurements.

While most /8 blocks contain traffic to a multitude of different services and organizations, it is still possible to discern some considerable differences in the RTTs even at this coarse level. Notably, many of the blocks with higher RTTs are clustered together. Figure 9a shows that for example blocks 58-62 to the left, 101, 102, 105, and 106 in the bottom, and 110-124 and 217-223 in the middle have high latency, with the median RTT in many cases exceeding 200 ms. This largely corresponds to blocks that are managed by the Asia Pacific Network Information Centre (APNIC) and African Network Information Centre (AFRINIC). While we acknowledge that the regional Internet registry (RIR) managing the block is a very imprecise measure of location, the high latency to these blocks indicates that they are indeed largely hosted in Asian and African regions without any nearby PoPs in the U.S. Furthermore, Figure 9b reveals that many of the same blocks also have a relatively high standard deviation. While the geographical distance inherently results in a high base latency, there are thus also other factors that cause large latency variations, further degrading the quality of experience for these remote connections.

Takeaway. Address blocks associated with Africa and Asia not only have higher latency, as expected from the distance, but also have a higher variation in latency. RTTs frequently exceeding 200 ms is likely to result in a poor QoE for any type of interactive applications connecting to these regions.

5.2 Variations Between Individual Subnets

While the different RTT distributions between /8 IPv4 blocks reveals some information about how the RTTs differ across different

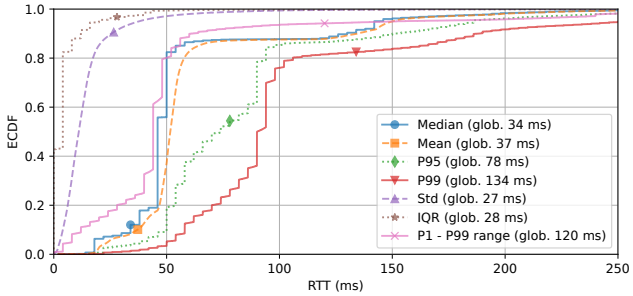


Figure 10: The distribution of several RTT metrics across individual external subnets. The markers along the lines and the values in the legend show the value of the RTT metrics when computed on all external RTTs, i.e. the RTT distribution shown in Figure 4b.

subnets, /8 blocks are very coarse grained and include many different subnets. To better understand how the RTT distribution within all the individual IPv4 /24 subnets and IPv6 /48 subnets vary, we therefore look at the distribution of several RTT metrics across all 78 657 external subnets with at least 100 RTT measurements.

Figure 10 shows that just like the overall RTT distribution itself, the distributions of RTT metrics computed for individual subnets are also long-tailed. In particular, we note a sharp increase in network latency for the top 13 % subnets, where the mean and median RTT suddenly jumps from around 60 ms to around 120 ms, typical of transcontinental connections. Looking at median and mean RTTs, almost 90 % of the subnets exceed the median and mean of the overall external RTTs, with the vast majority of the subnets instead having mean and medians in the 45–70 ms range. That the mean and median of the total RTT distribution is lower than for most individual subnets means that most of the traffic goes to the relatively few subnets with low latency, which is expected given that the major content providers utilize CDNs to reduce latency.

On the other hand, the 99th percentile and metrics related to variation show the opposite relationship between individual subnets and the RTT distribution as a whole. The lower variation within individual subnets is explained by the total RTT distribution being multi-modal, as the traffic is served from various different places, whereas individual subnets will largely be served from the same place and have more unimodal distributions. In particular, it is interesting to note that over 40 % of the subnets have an interquartile range (IQR) of 0, which means that at least the middlemost 50 % of their RTT measurements lie within the same 4 ms wide histogram bin. The lower variation also seems to result in lower 99th percentiles compared to the total RTT distribution for many subnets, with it being lower than 100 ms for 76 % of the subnets. This indicates that the latency tail in the overall RTT distribution is largely explained by a small fraction of traffic going to subnets with high RTTs overall, rather than all subnets necessarily having a long latency tail. However, the range between the 1st and 99th percentile shows that most subnets still have a considerable latency span, typically in a similar range as the mean and median RTT of the subnet. For around 75 % of the subnets, the P1-P99 span is at least 40 ms, and thereby exceeds the overall median and mean external RTT, and only 3.4 % of the subnets keep the 99th percentile below 50 ms.

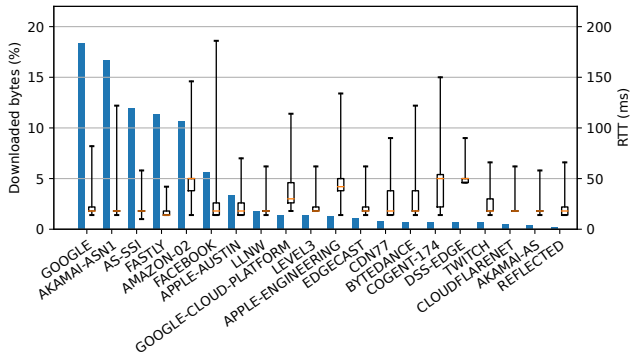
Takeaway. While a large part of the external latency variation can be explained by differences between subnets rather than variations within subnets, most subnets still have a thin latency tail. The latency for traffic that goes beyond the nearby CDNs quickly increases, which select monitoring of only the most popular services may miss.

5.3 RTT Distribution for Common ASes

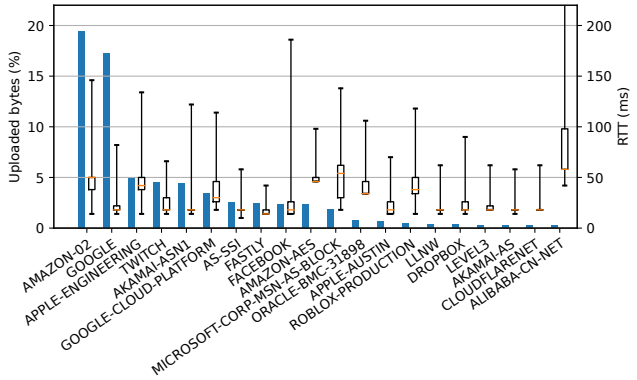
As evident from the large fraction of /8 blocks in Section 5.1 and individual subnets in Section 5.2 with higher RTTs than observed for most traffic, the amount of traffic is not evenly distributed across all subnets. To better reflect the latencies and latency variations subscribers typically experience, we focus on the network latency for the most common autonomous systems, translating the subnets to ASNs and ranking them by traffic in upload and download directions. As no routing information was collected during the measurement campaign, the subnet to ASN mapping was done during post processing based on data from RIPE NCC's route collector in Palo Alto, USA (RRC14) from the middle of the measurement period (2023-11-15). For the RTT distributions to be relevant, we filter the results to only include ASes with at least 1000 RTT samples, which rules out one AS, with 0.42 % of the upload traffic, from the top 20.

Figure 11 summarizes the RTT distribution for the 20 most popular ASes based on downlink and uplink traffic, respectively. The top 20 most popular ASes based on downlink traffic make up 89.5 % of all downlink traffic, while the top 20 ASes for uplink traffic cover 69.4 % of all uplink traffic, confirming that the vast majority of traffic goes towards just a handful of ASes. Note that the RTTs to the various ASes are always measured towards the AS, the separation between uplink and downlink traffic is only used to rank the ASes. Not surprisingly, a majority of the traffic comes from and goes to popular services and CDNs, like Google, Akamai, Netflix (AS-SSI), Fastly, Amazon, Facebook, and Apple. There is, however, a considerable difference in how downlink and uplink traffic are distributed across ASes, where AMAZON-2 and GOOGLE see substantially more uplink traffic compared to the other ASes.

To further illustrate how the RTT distribution can vary between ASes, we also provide a more detailed view of the RTT distributions for 4 ASes in Figure 12. For many of the ASes in Figure 11, such as GOOGLE, AKAMAI-ASN1, AS-SSI (Netflix), and FASTLY, the RTTs are mainly distributed in a narrow peak around 18 ms, exemplified by Figure 12a. This suggests that they are largely served from the Dallas data center. However, despite the overall very narrow distribution, there is still a thin but long tail, where the 99th percentile is often many times greater than the upper quartile. While not directly visible in the boxplots in Figure 11, the ASes with wider boxes, such as AMAZON-02, FACEBOOK, APPLE-AUSTIN, and GOOGLE-CLOUD-PLATFORM, have multi-modal distributions. For example, Figure 12b shows that AMAZON-02 has peaks around 18, 38, 50 and 94 ms, while Figure 12c shows peaks around 16 and 58 ms for FACEBOOK. This suggests that traffic to these ASes are handled at several different locations, with a fraction of it being served by the Dallas data center, and the rest at more remote locations.

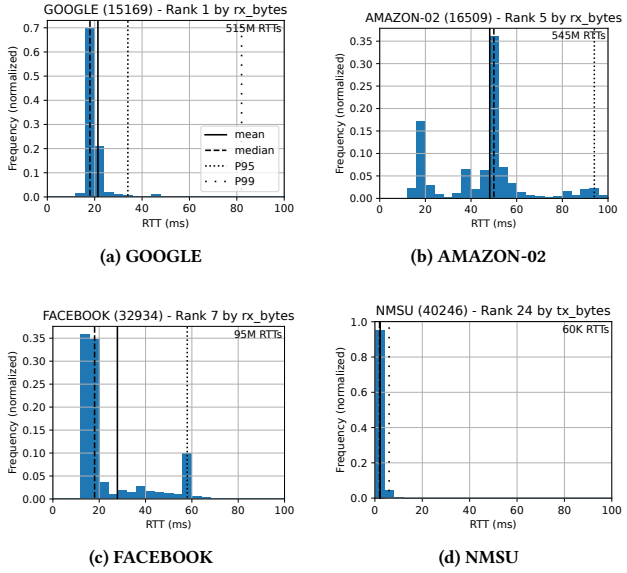


(a) Downlink



(b) Uplink

Figure 11: Fraction of traffic and RTT distribution for top 20 ASNs ranked by downlink and uplink traffic, respectively. The bar and the left y-axis show the fraction of all traffic that comes from or goes to the AS, while the boxplot and the right y-axis show the RTT distribution for the ASN. The orange line shows the median, and the whiskers show the 1st and 99th percentile.



(a) GOOGLE (b) AMAZON-02

(c) FACEBOOK (d) NMSU

Figure 12: RTT distribution for 4 select ASes.

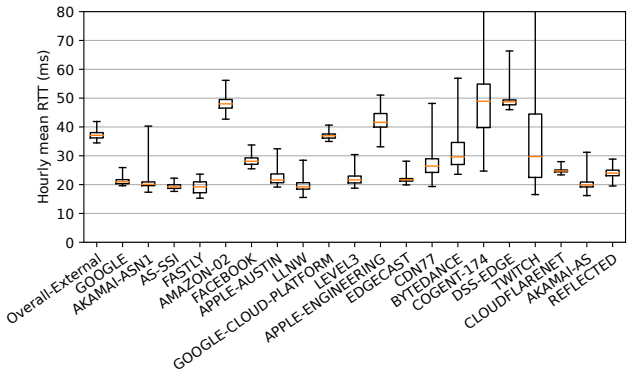


Figure 13: The distribution of the hourly mean RTT for the top 20 ASes with the most downlink traffic. The boxplot whiskers show the 1st and 99th percentile.

It is also interesting to note that the RTT distribution for NMSU (New Mexico State University) in Figure 12d differs from most other ASes. Almost all of the RTTs to NMSU are below 4 ms, i.e. within the first bin of the aggregated RTT histogram. This is explained by NMSU being geographically very close to the WISP network, and likely directly connected to the local data center in El Paso.

To also assess how stable the RTT distributions from the individual ASes are over time, we plot the distribution of the hourly mean RTT for each of the top 20 ASes by downlink traffic in Figure 13. As we noted in Section 4.4, the overall external RTT is relatively stable over time, with the hourly mean RTT typically staying between 36–38 ms as shown by the leftmost boxplot in Figure 13. While it is hardly surprising that the ASes with an overall narrow RTT distribution in Figure 11 are also stable over time, it is interesting to note that many of the ASes with wider overall RTT distributions still show little variation over time. This indicates that the traffic for e.g. AMAZON-02 and FACEBOOK is consistently split to different PoPs, rather than their multi-modal distributions being an effect of the PoPs handling the traffic changing over time. However, there are also a few ASes where the mean RTT shows large variations over time, like COGENT-174 and TWITCH. With the average latency to these ASes more than doubling during some periods, the quality of experience for the associated services may vary drastically.

While the latency to different ASes will obviously vary between different ISPs, monitoring the latency for popular ASes like this can help ISPs predict what quality of experience their customers can expect for various services and help inform future peering decisions. While JackRabbit Wireless already has reasonably low latency to the most popular services among their subscribers, reduced latency to some Amazon hosted services and a more stable latency to Twitch would still likely be desirable. Monitoring latency over time will

also allow ISPs to detect any potential service degradation and alert them to potential issues in the upstream networks, such as the latency spike to Amazon identified in Section 4.4.

Takeaway. The latency towards the most popular ASes generally show low variation and is stable over time, although several ASes have a mutli-modal latency distribution and a long latency tail is always present. Establishing latency profiles for popular ASes can help the ISP operator identify how to best improve their networks to benefit their subscriber base, by for example strategically peering or deploying network caches to improve latency for services that are currently under-performing.

6 Related Work

While there are numerous measurement studies analyzing various aspects of real world network performance on the Internet, only a fraction of them focus on network latency. Most measurement studies of Internet latency rely on various forms of active measurements, most commonly using periodic pings from various probes or small file downloads [4, 6, 8, 12, 25, 26, 28, 38, 44, 47].

In [6], Bozkurt et al. perform active measurements with curl to compare Internet latency with the theoretical minimum latency based on the speed of light, and find that due to inefficiencies in the Internet infrastructure the latency is one to two orders of magnitude higher than the limit imposed by the speed of light. Madanapalli et al. instead ping servers for several popular multiplayer games in [26], finding large differences in latency to the game servers from different ISPs based on their peering. Furthermore, ping measurements via the RIPE Atlas platform or Speedchecker network are used in [4, 12, 38, 44] to measure latency towards major cloud providers, generally finding that cloud providers lack presence in developing countries, resulting in high latencies there, and that the wireless last-mile access often contributes to a large portion of the end-to-end latency.

Similar approaches have also been used to study how the Covid pandemic [8] and Ukrainian war [25] impacted Internet performance, finding a measurable increase in latency at the start of these events, with [8] also finding latency to generally increase in the afternoon. Latency probes have also been used to assess the performance of different access technologies. Michel et al. uses a combination of ping probes and QUIC downloads to assess the performance of Starlink in [28], showing that it provides substantially lower latency than traditional geostationary satellite networks, but that the latency does increase a bit under load.

While active probing is flexible and simple to perform, measuring unloaded latency with ping-like measurements does not necessarily give an accurate view of the latency most user traffic will experience, as infrequent ping probes will not accurately capture queuing latency [9], or it may create excessive variations due to different route selections caused by load balancing [32]. To capture queuing delays, another set of works that employ active measurements focus on measuring latency during active download and upload tests [11, 18, 21].

For measuring latency under loaded conditions, [21] presents the web based Netalyzr platform, allowing users to run a large number of tests. From a global set of measurement runs, they find bufferbloat to be common and that it causes large increases in latency when the network is under load. Chirichella et al. also explore the presence of bufferbloat in [11] by downloading content with BitTorrent and extracting one-way latency information carried in the LEDBAT protocol header. Overall, they do not find bufferbloat to be a common problem for BitTorrent traffic, but that it does occur for a small fraction of traffic, for which it can add over one second of additional latency. Furthermore, the authors in [18] analyze a public dataset of active measurements from the M-Lab Network Diagnostic Tool, finding large RTT variations within flows, which is often a sign of bufferbloat, as well as considerable differences in RTT variations based on region.

Latency measurements performed during various forms of speed tests will capture latency under load, but the traffic load is artificial and to specific test servers, so it may not correspond to the latency users will experience for other Internet services. To better capture the latency of regular network traffic, some studies have, like us, used or complemented their data with passive latency measurements [1, 6, 14, 18, 27, 43, 47].

Two older studies [1, 27] collect full packet traces from a University Campus and an ISP, respectively, and then compute TCP RTTs offline. The authors in [1] mainly look at latency variations within flows, and like [18] find the span of RTTs within flows to be large. Maier et al. [27] only consider the TCP handshake RTT, but similarly to us break up the RTT into separate components for the internal and external networks. They find the internal RTT to generally be higher, and that WiFi contention can add several seconds of delay. More recent extensive passive measurement studies [14, 43] have been performed at PoliTO university, where they have used their tstat [15] tool to passively collect RTTs from TCP traffic. In [43] they present findings from a massive 5-year measurement campaign at an Italian ISP. Latency is only covered briefly, showing that the minimum external RTTs (excluding the latency in the access network) have decreased over time, and for some services they even observe sub-millisecond RTTs as the content is cached directly in the ISP PoPs. They also look at the within-flow variation, and unlike [18] find that it has decreased over time. In [14] they instead investigate how the Covid pandemic affected network traffic from their campus, and in particular their e-Learning platform. Once again, latency is only one of many aspects covered, this time showing that the distribution of minimum RTTs within each flow varies based on ISP and region, where wireless ISPs using 3G and 4G connections have considerably higher RTT variations.

In addition to analyzing M-Lab data, the authors in [18] capture traffic from an ISP access link and passively infer the RTT on the access-network during the TCP handshake, showing it to correlate with traffic load. The authors in [6] confirm their active curl measurements with a 24-hour passive capture from a CDN, collecting RTTs from the TCP handshakes between the CDN servers and clients. Yuan et al. [47] primarily use passive measurements from within a crowdsourced video streaming app, complemented with traceroute probes, to compare latency across 4G, 5G standalone (SA) and 5G non-standalone (NSA) links, finding 5G SA to generally provide lower latency.

Our measurement study is not the most extensive, covering a one month period at a smaller ISP. However, it serves as a complement to previous work by analyzing RTT variations of both the internal and external network segments in a WISP setting, looking into how

the latency varies over time and across various subnets. Through the extended epping tool, our work also contributes a means for ISPs to carry out continuous lightweight latency monitoring. Many of our results are consistent with previous findings. While we unlike [27] no longer find the internal RTTs to be overall higher than the external ones, we still find a large internal latency tail, which 15 years later can still likely be attributed to the last-hop WiFi access. Like [6, 8] we find that the latency increases in the afternoon and evening, at least on the internal network, and also find a correlation between latency and traffic load like [18]. While we did not look directly at RTT variations within flows, the relatively narrow RTT distributions we found within individual external subnets would suggest much smaller within-flow variations than reported in [1] and [18]. However, [43] notes that the external within-flow variation has decreased over the years, and the large internal latency tail we observed may still result in large end-to-end RTT variations in flows.

7 Limitations

Collecting latency data in an aggregated format drastically reduces the footprint of the collected data and makes it feasible to continuously monitor latency, but the aggregation also results in some loss of information and granularity. One notable limitation with our aggregation is the coarse 4 ms granularity of the histogram the RTTs are collected in, as well as large RTTs effectively being truncated to the 1000 ms range of the histograms. As mentioned in Section 2.5, a more complex implementation could use a histogram with non-uniform bins to more accurately capture low RTTs and extend the maximum range of the histogram without requiring a large increase in number of bins. However, we believe that for many use cases a 4 ms resolution is still sufficient to inform the network operator about the type of QoE the network delivers, and RTTs above 1000 ms will provide a poor experience for any interactive application, regardless of the actual value.

Furthermore, we only collect the data at subnet granularity and can therefore not examine per-flow information, such as RTT variations within flows, which may be useful to provide more detailed insights on the QoE and infer the presence of bufferbloat. We note than for an ISP it may be desirable to aggregate the internal traffic on a per-IP level (i.e. a /32 subnet for IPv4) rather than the /24 subnets we used for all traffic, to allow associating the data with individual subscribers. Indeed, this is what JackRabbit Wireless normally does to identify subscribers who may suffer from a poor QoE. However, for our study we explicitly wanted to avoid collecting information that could be tied to individuals, and therefore only consider the internal network as a whole. Future work could improve on our work by tracking relevant per-flow metrics in the kernel but still output them in an aggregated format to avoid disclosing privacy sensitive information.

We also note that while histograms scale well to a large number of RTT measurements, in our setup there were many subnets with little traffic that only recorded a handful of RTTs. For these subnets with few RTT measurements it would be more efficient to keep the individual RTT values instead of storing them in a histogram. The significance of this issue will likely decrease for networks processing larger amounts of traffic, and could also be partially counteracted by using coarser aggregation than what we used, such as larger subnets or longer aggregation periods. None the less, a better method to aggregate data from these low traffic subnets would still be beneficial. Despite these inefficiencies, however, our entire dataset consists of only 11.8 GB of compressed JSON data, in contrast to the roughly 36 TB that would be required for a corresponding packet trace with TCP headers of all 356 billion packets we observed during the measurement period. Through our analysis, we have demonstrated that this aggregated data still allows for various forms of useful analysis. By releasing the source code, we also enable other researchers and engineers to improve upon our work and add features for their specific use cases.

As for the reported findings in the data, we note that the scope of our study is limited to a single WISP over a single month. As such, the generalizability of the findings are limited. The measured RTT values in various parts of the networks will obviously vary between different ISPs. However, we note that many of our findings are still consistent with what has been reported in related work, and we expect that many of the more general findings will also apply to many other ISP networks. This includes for example the high occurrence of TCP timestamps, the long and variable latency tail in the last mile access, the multi-modal distribution of external network latency, and the latency towards major ASes being largely stable over time. Ultimately, the analysis here primarily serves as an example of what ISPs and other network operators can learn about their networks through continuous passive monitoring of network latency.

8 Conclusion

In this work we present an extension for epping that aggregates RTT measurements per subnet, and demonstrate the feasibility of continuously monitoring network latency in live networks by deploying it at a WISP. Our measurement solution only imposes around a 1 % CPU overhead while collecting over 9 billion RTT measurements across a month. The latency monitoring reveals a large latency tail in the internal network, which increases during the busy periods in the evening, likely caused by WiFi networks inside subscriber premises. Furthermore we find large differences in the external latency, identifying different latency profiles for several popular ASes. Overall, our hope is that by providing our extension to epping for collecting aggregated latency measurements as well as the dataset used in this paper, we can help researchers and network operators to further study how latency varies in their networks and share their findings with the community.

Acknowledgments

This work has been partly funded by the Knowledge Foundation of Sweden (KKS), grants 20200067 and 20220072, and by Red Hat Research. The authors extend their gratitude to Jesper Dangaard Brouer at Cloudflare for his support with implementing the epping tool and troubleshooting performance issues. The authors also wish to thank the anonymous reviewers for their detailed feedback that greatly helped improve the paper.

A Ethics

Although the core of the study is about capturing and analyzing real user traffic from an ISP, the dataset does not contain any identification of individual connections. Specifically, it does not contain any full IP addresses. All data is aggregated per subnet and over a time period (10s). Hence it is not possible to directly link the measurements to individual subscribers; only the subscriber base as a whole. This study has been submitted to and approved by the ethical board at the Faculty of Health, Science and Technology at Karlstad University.

References

[1] Jay Aikat, Jasleen Kaur, F. Donelson Smith, and Kevin Jeffay. 2003. Variability in TCP Round-Trip Times. In *Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement (IMC '03)*. Association for Computing Machinery, New York, NY, USA, 279–284. https://doi.org/10.1145/948205.948241

[2] Tom Barbette, Erfan Wu, Dejan Kostić, Gerald Q. Maguire, Panagiotis Papadimitratos, and Marco Chiesa. 2022. Cheetah: A High-Speed Programmable Load-Balancer Framework with Guaranteed per-Connection-Consistency. *IEEE/ACM Transactions on Networking* 30, 1 (Feb. 2022), 354–367. https://doi.org/10.1109/TNET.2021.3113370

[3] Miguel Barreda-Ángeles, Ioannis Arapakis, Xiao Bai, B. Barla Cambazoglu, and Alexandre Pereda-Baños. 2015. Unconscious Physiological Effects of Search Latency on Users and Their Click Behaviour. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '15)*. Association for Computing Machinery, New York, NY, USA, 203–212. https://doi.org/10.1145/2766462.2767719

[4] Zachary S. Bischof, John P. Rula, and Fabián E. Bustamante. 2015. In and Out of Cuba: Characterizing Cuba's Connectivity. In *Proceedings of the 2015 Internet Measurement Conference (IMC '15)*. Association for Computing Machinery, New York, NY, USA, 487–493. https://doi.org/10.1145/2815675.2815718

[5] David Borman, Robert T. Braden, Van Jacobson, and Richard Scheffenegger. 2014. *TCP Extensions for High Performance*. Request for Comments RFC 7323. Internet Engineering Task Force. https://doi.org/10.17487/RFC7323

[6] Ilker Nadi Bozkurt, Anthony Aguirre, Balakrishnan Chandrasekaran, P. Brighten Godfrey, Gregory Laughlin, Bruce Maggs, and Ankit Singla. 2017. Why Is the Internet so Slow?!. In *Passive and Active Measurement*, Mohamed Ali Kaafar, Steve Uhlig, and Johanna Amann (Eds.). Springer International Publishing, Cham, 173–187. https://doi.org/10.1007/978-3-319-54328-4_13

[7] Francesco Bronzino, Paul Schmitt, Sara Ayoubi, Guilherme Martins, Renata Teixeira, and Nick Feamster. 2019. Inferring Streaming Video Quality from Encrypted Traffic: Practical Models and Deployment Experience. *Proc. ACM Meas. Anal. Comput. Syst.* 3, 3, Article 56 (Dec. 2019), 25 pages. https://doi.org/10.1145/3366704

[8] Massimo Candela, Valerio Luconi, and Alessio Vecchio. 2020. Impact of the COVID-19 Pandemic on the Internet Latency: A Large-Scale Study. *Computer Networks* 182 (2020), 107495. https://doi.org/10.1016/j.comnet.2020.107495

[9] Pellegrino Casoria, Dario Rossi, Jordan Augé, Marc-Olivier Buob, Timur Friedman, and Antonio Pescapé. 2014. Distributed Active Measurement of Internet Queuing Delays. In *Passive and Active Measurement*, Michalis Faloutsos and Aleksandar Kuzmanovic (Eds.). Springer International Publishing, Cham, 278–280. https://doi.org/10.1007/978-3-319-04918-2_31

[10] Stuart Cheshire. 2001. It's the Latency, Stupid. http://www.stuartcheshire.org/rants/Latency.html

[11] Chiara Chirichella and Dario Rossi. 2013. To the Moon and Back: Are Internet Bufferbloat Delays Really That Large?. In *2013 Proceedings IEEE INFOCOM*. 3297–3302. https://doi.org/10.1109/INFCOM.2013.6567154

[12] The Khang Dang, Nitinder Mohan, Lorenzo Corneo, Aleksandr Zavodovski, Jörg Ott, and Jussi Kangasharju. 2021. Cloudy with a Chance of Short RTTs: Analyzing Cloud Connectivity in the Internet. In *Proceedings of the 21st ACM Internet Measurement Conference (IMC '21)*. Association for Computing Machinery, New York, NY, USA, 62–79. https://doi.org/10.1145/3487552.3487854

[13] Italo Epicoco, Catiuscia Melle, Massimo Cafaro, Marco Pulimeno, and Giuseppe Morleo. 2020. UDDSketch: Accurate Tracking of Quantiles in Data Streams. *IEEE access : practical innovations, open solutions* 8 (2020), 147604–147617. https://doi.org/10.1109/ACCESS.2020.3015599

[14] Thomas Favale, Francesca Soro, Martino Trevisan, Idilio Drago, and Marco Mellia. 2020. Campus Traffic and E-Learning during COVID-19 Pandemic. *Computer Networks* 176 (2020), 107290. https://doi.org/10.1016/j.comnet.2020.107290

[15] Alessandro Finamore, Marco Mellia, Michela Meo, Maurizio M. Munafo, Politecnico Di Torino, and Dario Rossi. 2011. Experiences of Internet Traffic Monitoring with Tstat. *IEEE Network* 25, 3 (May 2011), 8–14. https://doi.org/10.1109/MNET.2011.5772055

[16] Ilya Grigorik. 2012. Latency: The New Web Performance Bottleneck - Igvita.Com. https://www.igvita.com/2012/07/19/latency-the-new-web-performance-bottleneck/

[17] Heinrich Hartmann and Theo Schlossnagle. 2020. Circllhist - A Log-Linear Histogram Data Structure for IT Infrastructure Monitoring. *ArXiv* abs/2001.06561 (2020), 16. https://arxiv.org/pdf/2001.06561.pdf

[18] Toke Høiland-Jørgensen, Bengt Ahlgren, Per Hurtig, and Anna Brunstrom. 2016. Measuring Latency Variation in the Internet. In *Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies (CoNEXT '16)*. Association for Computing Machinery, New York, NY, USA, 473–480. https://doi.org/10.1145/2999572.2999603

[19] Toke Høiland-Jørgensen, Per Hurtig, and Anna Brunstrom. 2015. The Good, the Bad and the WiFi: Modern AQMs in a Residential Setting. *Computer Networks* 89 (2015), 90–106. https://doi.org/10.1016/j.comnet.2015.07.014

[20] Internet Assigned Numbers Authority. 2024. Protocol Numbers. https://www.iana.org/assignments/protocol-numbers/protocol-numbers.xhtml

[21] Christian Kreibich, Nicholas Weaver, Boris Nechaev, and Vern Paxson. 2010. Netalyzr: Illuminating the Edge Network. In *Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement (IMC '10)*. Association for Computing Machinery, New York, NY, USA, 246–259. https://doi.org/10.1145/1879141.1879173

[22] LibreQoE, LLC. 2023. LibreQoS - Fast, Flexible QoE for Smart ISPs. https://libreqos.io/

[23] Hyoyoung Lim, Seonwoo Kim, Jackson Sippe, Junseon Kim, Greg White, Chul-Ho Lee, Eric Wustrow, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. 2022. A Fresh Look at ECN Traversal in the Wild. https://doi.org/10.48550/arXiv.2208.14523 arXiv:2208.14523

[24] Chang Liu, Byungchul Tak, and Long Wang. 2024. Understanding Performance of eBPF Maps. In *Proceedings of the ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions (eBPF '24)*. Association for Computing Machinery, New York, NY, USA, 9–15. https://doi.org/10.1145/3672197.3673430

[25] Valerio Luconi and Alessio Vecchio. 2023. Impact of the First Months of War on Routing and Latency in Ukraine. *Computer Networks* 224 (2023), 109596. https://doi.org/10.1016/j.comnet.2023.109596

[26] Sharat Chandra Madanapalli, Hassan Habibi Gharakheili, and Vijay Sivaraman. 2022. Know Thy Lag: In-network Game Detection and Latency Measurement. In *Passive and Active Measurement*, Oliver Hohlfeld, Giovane Moura, and Cristel Pelsser (Eds.). Springer International Publishing, Cham, 395–410. https://doi.org/10.1007/978-3-030-98785-5_17

[27] Gregor Maier, Anja Feldmann, Vern Paxson, and Mark Allman. 2009. On Dominant Characteristics of Residential Broadband Internet Traffic. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement (IMC '09)*. Association for Computing Machinery, New York, NY, USA, 90–102. https://doi.org/10.1145/1644893.1644904

[28] François Michel, Martino Trevisan, Danilo Giordano, and Olivier Bonaventure. 2022. A First Look at Starlink Performance. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC '22)*. Association for Computing Machinery, New York, NY, USA, 130–136. https://doi.org/10.1145/3517745.3561416

[29] Seraj Al Mahmud Mostafa, Mike Wittie, and Utkarsh Goel. 2020. Does More Bandwidth Really Not Matter (Much)?. In *Proceedings of the 13th EAI International Conference on Mobile Multimedia Communications*. EAI. https://doi.org/10.4108/eai.27-8-2020.2299825

[30] Randall Munroe. 2006. Map of the Internet. https://xkcd.com/195/

[31] Changhua Pei, Youjian Zhao, Guo Chen, Ruming Tang, Yuan Meng, Minghua Ma, Ken Ling, and Dan Pei. 2016. WiFi Can Be the Weakest Link of Round Trip Network Latency in the Wild. In *IEEE INFOCOM 2016 - the 35th Annual IEEE International Conference on Computer Communications*. 1–9. https://doi.org/10.1109/INFOCOM.2016.7524396

[32] Cristel Pelsser, Luca Cittadini, Stefano Vissicchio, and Randy Bush. 2013. From Paris to Tokyo: On the Suitability of Ping to Measure Latency. In *Proceedings of the 2013 Conference on Internet Measurement Conference (IMC '13)*. Association for Computing Machinery, New York, NY, USA, 427–432. https://doi.org/10.1145/2504730.2504765

[33] Constantin Sander, Ike Kunze, Leo Blöcher, Mike Kosek, and Klaus Wehrle. 2023. ECN with QUIC: Challenges in the Wild. In *Proceedings of the 2023 ACM on Internet Measurement Conference (IMC '23)*. Association for Computing Machinery, New York, NY, USA, 540–553. https://doi.org/10.1145/3618257.3624821

[34] Koen De Schepper and Bob Briscoe. 2023. *The Explicit Congestion Notification (ECN) Protocol for Low Latency, Low Loss, and Scalable Throughput (L4S)*. Request for Comments RFC 9331. Internet Engineering Task Force. https://doi.org/10.17487/RFC9331

[35] Satadal Sengupta, Hyojoon Kim, and Jennifer Rexford. 2022. Continuous In-Network Round-Trip Time Monitoring. In *Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22)*. Association for Computing Machinery, New York, NY, USA, 473–485. https://doi.org/10.1145/3544216.3544222

[36] Shawn Ostermann. 2013. Tcptrace. https://github.com/blitz/tcptrace

[37] Jaykumar Sheth, Vikram Ramanna, and Behnam Dezfouli. 2021. FLIP: A Framework for Leveraging eBPF to Augment WiFi Access Points and Investigate Network Performance. In *Proceedings of the 19th ACM International Symposium on*

Mobility Management and Wireless Access (MobiWac '21). Association for Computing Machinery, New York, NY, USA, 117–125. https://doi.org/10.1145/3479241.3486700

[38] Rachee Singh, Arun Dunna, and Phillipa Gill. 2018. Characterizing the Deployment and Performance of Multi-CDNs. In *Proceedings of the Internet Measurement Conference 2018 (IMC '18)*. Association for Computing Machinery, New York, NY, USA, 168–174. https://doi.org/10.1145/3278532.3278548

[39] Joel Sommers, Nolan Rudolph, and Ramakrishnan Durairajan. 2023. Schooling NOOBs with eBPF. In *Proceedings of the 1st Workshop on eBPF and Kernel Extensions (eBPF '23)*. Association for Computing Machinery, New York, NY, USA, 21–27. https://doi.org/10.1145/3609021.3609302

[40] Kaixin Sui, Mengyu Zhou, Dapeng Liu, Minghua Ma, Dan Pei, Youjian Zhao, Zimu Li, and Thomas Moscibroda. 2016. Characterizing and Improving WiFi Latency in Large-Scale Operational Networks. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '16)*. Association for Computing Machinery, New York, NY, USA, 347–360. https://doi.org/10.1145/2906388.2906393

[41] Simon Sundberg, Anna Brunstrom, Simone Ferlin-Reiter, Toke Høiland-Jørgensen, and Jesper Dangaard Brouer. 2023. Efficient Continuous Latency Monitoring with eBPF. In *Passive and Active Measurement*, Anna Brunstrom, Marcel Flores, and Marco Fiore (Eds.). Springer Nature Switzerland, Cham, 191–208. https://doi.org/10.1007/978-3-031-28486-1_9

[42] Gil Tene. n.d.. HdrHistogram: A High Dynamic Range Histogram. https://hdrhistogram.github.io/HdrHistogram/

[43] Martino Trevisan, Danilo Giordano, Idilio Drago, Maurizio Matteo Munafò, and Marco Mellia. 2020. Five Years at the Edge: Watching Internet from the ISP Network. *IEEE/ACM Transactions on Networking* 28, 2 (April 2020), 561–574. https://doi.org/10.1109/TNET.2020.2967588

[44] Opeoluwa Victor Babasanmi and Josiah Chavula. 2022. Measuring Cloud Latency in Africa. In *2022 IEEE 11th International Conference on Cloud Networking (CloudNet)*. 61–66. https://doi.org/10.1109/CloudNet55617.2022.9978896

[45] Sara Vlahovic, Mirko Suznjevic, and Lea Skorin-Kapov. 2019. The Impact of Network Latency on Gaming QoE for an FPS VR Game. In *2019 Eleventh International Conference on Quality of Multimedia Experience (QoMEX)*. 1–3. https://doi.org/10.1109/QoMEX.2019.8743193

[46] Hengchao Wang, Xu Zhang, Hao Chen, Yiling Xu, and Zhan Ma. 2022. Inferring End-to-End Latency in Live Videos. *IEEE Transactions on Broadcasting* 68, 2 (June 2022), 517–529. https://doi.org/10.1109/TBC.2021.3071060

[47] Xinjie Yuan, Mingzhou Wu, Zhi Wang, Yifei Zhu, Ming Ma, Junjian Guo, Zhi-Li Zhang, and Wenwu Zhu. 2022. Understanding 5G Performance for Real-World Services: A Content Provider's Perspective. In *Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22)*. Association for Computing Machinery, New York, NY, USA, 101–113. https://doi.org/10.1145/3544216.3544219